TECHNICAL & ENGINEERING REPORT

URSC — A Modular Engine for Long-Term LEO Trajectory Prediction and Orbit Determination

Architecture, force models, numerical methods, validation, and development roadmap

System: Java 17, Maven, **zero runtime dependencies** (JUnit 5 for tests only)

Reference architecture: Orekit (independent clean-room implementation)

Capabilities: Cowell numerical & DSST semi-analytical propagation; batch least-squares OD; full IAU-2006/2000B frames; CLI toolchain

Validation: Sentinel-3A/3B POEORB precise orbits (16 Oct 2024)

Test suite: 37 unit tests, all passing

Abstract. URSC is a from-scratch, dependency-free Java library for propagating and determining LEO satellite trajectories over long arcs. Every force model, integrator, atmosphere, frame model and output type is a pluggable component behind a small set of interfaces, assembled through a fluent configuration builder. The engine implements a complete perturbation stack (EGM96 geopotential to d/o 70, luni-solar third body, NRLMSISE-00 drag, conical SRP, solid/ocean tides, relativity), two propagators (adaptive Cowell and a numerically-averaged DSST mean-element method), a rigorous GCRF ↔ ITRF transformation, event detection, and a batch least-squares estimator. This report describes the software architecture, the implementation of each physical and numerical component, the validation against independent Sentinel-3 precise orbits (radial/cross-track held to metres over 24 h; batch fit reducing residuals ~8×), the error budget, and a prioritised roadmap of remaining work.

Contents

1 Objectives and design principles
2 Software architecture
3 Force models (implementation)
4 Reference frames and time scales
5 Numerical methods: integration & events
6 Propagators: Cowell and DSST
7 Orbit determination (batch least squares)
8 Input/output and configuration
9 Validation methodology and results
10 Verification: the automated test suite
11 Current limitations
12 Roadmap — what remains and how to improve
13 Appendix: constants & CLI reference

1 Objectives and design principles

URSC was built to determine and predict long-term LEO trajectories with a fully modular, inspectable, dependency-free codebase, using Orekit as an architectural reference but sharing no code.

Four principles drive the design:

- **Modularity.** Every perturbation is a `ForceModel`; every atmosphere an `Atmosphere`; every integrator an `ODEIntegrator`; every frame realisation a selectable model. Components are composed, not hard-wired, so a force or a model can be added or swapped without touching callers.
- **Configurability.** Every physical variable (degree/order, coefficients, areas, masses, tolerances, space weather) is an explicit input, set in code via a fluent builder or in a `.properties` scenario file.
- **Zero runtime dependencies.** The entire stack — linear algebra, special functions, ephemerides, the NRLMSISE-00 thermosphere, frame transformations — is implemented in-house. Bundled reference data (gravity coefficients, nutation series, tide tables) ship as compressed classpath resources.
- **Auditability.** Each model cites its governing reference (Montenbruck & Gill, IERS-2010 Conventions, SOFA) and is checked against published values in the test suite.

2 Software architecture

2.1 Package map

Table 1. Top-level packages under `org.ursc.trajectory`.

Package	Responsibility	Key types
`math`	Vectors, rotation matrices, constants	Vector3D, RotationMatrix, Constants
`time`	Time scales & epochs	AbsoluteDate, TimeScalesFactory, UTCScale
`frames`	GCRF/ITRF, IAU-2006/2000B, EOP	FramesFactory, Iau2006, Transform, EopProvider
`bodies`	Ephemerides, Earth ellipsoid	CelestialBodyFactory, OneAxisEllipsoid
`orbits`	Element sets & conversions	Keplerian/Equinoctial/CartesianOrbit, PVCoordinates
`forces`	All perturbations	ForceModel, gravity/*, drag/*, radiation/*, Relativity
`ode`	Integrators	DormandPrince54Integrator, ClassicalRungeKutta
`propagation`	Propagators, events, sampling	NumericalPropagator, semianalytical/DSSTPropagator
`estimation`	Orbit determination	BatchLeastSquares, PositionObservation
`config`	Scenario assembly	PropagationConfig, ScenarioLoader
`io`	External data	PoeOrbReader, CSV writers, TLE parser
`analysis`, `app`	Decay/compare tools, CLI	DecayPredictor, ModelComparison, Cli + commands

2.2 Core abstractions

The whole engine turns on a few small interfaces:

- `ForceModel` — `Vector3D acceleration(SpacecraftState)`. Every perturbation implements this one method; the propagator simply sums them.
- `Atmosphere` — `double getDensity(date, rBodyFixed)`, so the drag force is agnostic to the density model (exponential / Harris–Priester / NRLMSISE-00).
- `ODEIntegrator` — a single adaptive or fixed step; the propagator drives it.
- `SampledPropagator` — the common surface (step handlers, events, output type) that *both* the numerical and DSST propagators implement, so analysis tools accept either engine interchangeably.

Configuration is assembled with a fluent builder, `PropagationConfig`, which gathers the initial state, integrator, force toggles and output type and emits a ready propagator. The same configuration object is reused by the CLI, the comparison tool, the decay predictor and the estimator.

3 Force models (implementation)

All forces return an acceleration in the state's inertial frame and are summed by the propagator. Table 2 is the implementation index; the physical derivations and governing equations are given in the companion *Physics Report*. Constants are listed in the Appendix.

Table 2. Implemented force models, with class, algorithm and data source.

Model	Class	Algorithm / standard	Data / constants
Central + geopotential	`SphericalHarmonicGravity`	Cunningham–Gottlieb recursion (M&G §3.2.4)	EGM96 d/o 70, bundled gzip resource
Point-mass central	`NewtonianAttraction`	$-\mu r/r^3$	μ = 3.986004418e14
Third body	`ThirdBodyAttraction`	Direct + indirect term	Sun/Moon analytical series
Atmospheric drag	`DragForce` + `Atmosphere`	Cannonball, co-rotating v_{rel}	NRLMSISE-00 / HP / exponential
Thermosphere	`NRLMSISE00Atmosphere`	`gtd7d` (verified port)	SpaceWeatherProvider (F10.7, Ap)
Solar radiation pressure	`SolarRadiationPressure`	Conical umbra/penumbra	P_0 = 4.56e-6 N/m²
Solid Earth tides	`SolidTides`	IERS-2010 Step 1 (deg 2,3,+k^*)	Love numbers, Table 6.3
Ocean tides	`OceanTides`	IERS-2010 eq 6.15, Doodson args	FES2004, 8 constituents, d/o 4
Relativity	`Relativity`	Schwarzschild (+opt. Lense–Thirring)	IERS-2010 eq 10.12, $\beta=\gamma=1$
Earth albedo / IR	`EarthRadiationPressure`	Knocke–Ries (available)	Reflectivity/emissivity rings
Empirical accel.	`EmpiricalAcceleration`	RTN constant/periodic terms	User / estimated

> **Normalisation & frames.** EGM96 coefficients are stored fully normalised and de-normalised on load via the standard factor; the gravity acceleration is evaluated in ITRF (where the field is static) and rotated to GCRF each call. Drag likewise maps the inertial position to ITRF to read the density at the correct geodetic point. These per-call frame transforms are what make a correct GCRF ↔ ITRF model (§4) essential to overall accuracy.

3.1 Drag — the accuracy-limiting force

Drag uses the cannonball model with the relative velocity taken against a co-rotating atmosphere ($v_{rel} = v - \boldsymbol{\omega}_{\oplus} \times r$). Density comes from the pluggable atmosphere; with NRLMSISE-00 the drivers (daily F10.7, 81-day average F10.7a, daily Ap) are supplied by a `SpaceWeatherProvider` that is either constant or read from a CelesTrak CSSI file. The ballistic grouping $C_D A/m$ is the natural estimation parameter (§7, §9).

4 Reference frames and time scales

Propagation is performed in GCRF (pseudo-inertial). The GCRF ↔ ITRF transform has two selectable realisations: a fast sidereal-only `SIMPLE` model (GMST rotation, IAU-82 polynomial) and the default full `IAU_2006` model implemented in `Iau2006`:

- **Precession** — IAU-2006 Fukushima–Williams angles;
- **Nutation** — IAU-2000B 77-term luni-solar series (~1 mas), coefficients in `Nutation2000BData`, ported from the SOFA reference;
- **Earth rotation** — GAST (GMST + equation of the equinoxes) with UT1 from the EOP provider;
- **Polar motion** — small-angle wobble matrix W(x_p, y_p).

EOP (UT1−UTC, polar motion) are injected through an `EopProvider` — either `ConstantEop` or a file-backed reader. Time scales (`TimeScalesFactory`) cover TAI, TT = TAI + 32.184 s, UTC with the full 1972–2017 leap-second table, GPS, and UT1. `AbsoluteDate` stores continuous TAI seconds from J2000, so epoch differences are exact across leap seconds — a subtle correctness point for long arcs. The IAU-2006 chain reproduces SOFA to ~10^{-13} rad in the test suite.

5 Numerical methods: integration and events

5.1 Dormand–Prince 5(4)

The recommended integrator is the adaptive **Dormand–Prince 5(4)** embedded Runge–Kutta pair (DOPRI5): a seven-stage, first-same-as-last (FSAL) scheme. The fifth-order solution is propagated and the difference against the embedded fourth-order solution drives per-step error control:

$$\hat{\vec{y}}_{k+1} = \vec{y}_k + h\sum_i b_i \vec{k}_i, \qquad \|\vec{e}\| = \|\hat{\vec{y}}_{k+1} - \vec{y}^{*}_{k+1}\|, \qquad h_{\text{new}} = h\, S\Big(\frac{\text{tol}}{\|\vec{e}\|}\Big)^{1/5} \tag{1}$$

with step scaling by a safety factor (0.9), bounded growth/shrink factors [0.2, 5], and absolute/relative tolerances (validation used $10^{-7}/10^{-9}$). The step automatically shrinks near perigee and in denser atmosphere and grows where dynamics are smooth. A fixed-step classical RK4 is also provided. Dense (cubic-Hermite) interpolation yields output on an arbitrary user grid without re-integrating.

5.2 Event detection

Events are located by a g-function sign change with bracketing and bisection refinement, supporting apside, node, altitude (re-entry) and eclipse detectors. This is what lets the decay/lifetime tools stop cleanly at a re-entry altitude and lets SRP/eclipse analyses find shadow boundaries.

6 Propagators: Cowell and DSST

6.1 Numerical (Cowell)

`NumericalPropagator` integrates the full osculating Cartesian state (*r*, *v*) under the summed forces using the chosen integrator, with step handlers, dense output and event detection. It is the high-accuracy workhorse used for the validation and OD below.

6.2 Semi-analytical (DSST, mean elements)

For multi-year and lifetime studies, resolving every ~100-minute revolution with Cowell is expensive. The `DSSTPropagator` implements a **numerically-averaged** mean-element method: it propagates the six mean equinoctial elements (a, e_x, e_y, h_x, h_y, λ) by orbit-averaging the variational equations:

$$\left\langle \frac{d\mathcal{E}}{dt} \right\rangle = \frac{1}{2\pi}\int_0^{2\pi} \frac{\partial \mathcal{E}}{\partial \vec{v}} \cdot \vec{\gamma}\,(1 - e\cos E)\, dE, \qquad \dot{\lambda} \to \dot{\lambda} + n(a) \tag{2}$$

At each large step, the osculating state is built at *N* points (default 24) around the current mean orbit; the perturbing acceleration (sum of the *same* `ForceModel`s minus the two-body term) is evaluated at each; the element rates `∂E/∂v·γ` are formed by finite difference of the equinoctial elements with respect to velocity (step 10^{-3} m/s); and these are averaged with the `(1−e cos E)` weight, with the mean longitude additionally advancing at the mean motion

n(a). The result is **mean** elements, stepped in hours-to-a-day — roughly **20× faster** than Cowell for long runs, reusing every force model. Short-period recovery (mean → osculating) is a documented follow-up.

7 Orbit determination (batch least squares)

`BatchLeastSquares` estimates a parameter vector — the initial state [*r*, *v*] and optionally the drag coefficient C_D — by fitting the propagated trajectory to position observations through Gauss–Newton iteration. The design matrix (the partials of computed position with respect to the parameters) is formed by **finite differences**, so the estimator works with *any* force model and either propagation engine — no analytical variational equations are required. The scaled normal equations with small Levenberg–Marquardt damping are

$$\left(\tilde{J}^{\mathrm{T}}\tilde{J} + \mu\,\mathrm{diag}(\tilde{J}^{\mathrm{T}}\tilde{J})\right)\Delta\tilde{\vec{p}} = \tilde{J}^{\mathrm{T}}\vec{r}, \qquad \tilde{J}_{ij} = \frac{\partial\vec{\rho}_i}{\partial p_j}\,s_j \tag{3}$$

where ρ_i is the residual at observation *i*, s_j the per-parameter scale (position 1, velocity 10^{-3}, C_D 10^{-2}) that conditions the problem, and μ a tiny damping term. The step is accepted with step-halving backtracking (up to 6 contractions) and the iteration converges when the relative RMS improvement falls below 10^{-4}. The linear system is solved by Gaussian elimination with partial pivoting. Observations are `PositionObservation`s (GCRF, metres), here sampled from the POEORB precise orbit.

> **Interface.** The estimator is driven through a `PropagatorFactory` lambda that rebuilds a seeded propagator from a parameter vector, so the same code fits either the Cowell or DSST engine, with any subset of forces.

8 Input/output and configuration

- **POEORB reader** (`PoeOrbReader`) — parses CCSDS/Earth-Explorer `AUX_POEORB *.EOF` precise-orbit files: time-tagged ITRF state vectors plus the carried UT1−UTC offset. Drives both `validate` and `fit`.
- **Scenario files** (`ScenarioLoader`) — `.properties` files where each force is an independent toggle with its own parameters; selects the propagator (numerical/dsst), integrator, orbit, spacecraft, output and re-entry event.
- **CSV writers** — ephemeris export, including ground track.
- **TLE parser** — approximate osculating seeding (no SGP4 mean-to-osculating recovery yet).
- **CLI** (`Cli`) — zero-dependency sub-command dispatcher: `propagate`, `decay`, `compare`, `tle`, `validate`, `fit`, with ASCII plots and RTN error reporting in the terminal.

9 Validation methodology and results

9.1 Method

The `validate` command seeds the propagator from the first POEORB state (ITRF → GCRF with the file's own UT1), integrates forward under the full model with no fitting, and differences the propagated position against the precise orbit at each epoch, decomposing the error into radial, along-track and cross-track components. The `fit` command then runs the batch estimator against the same arc. Run configuration: mass 1130 kg, area 8 m², C_D = 2.2, C_R = 1.3, F10.7/F10.7a/Ap = 220/205/12, gravity 30×30, DP54 tolerances 10^{-7}/10^{-9}.

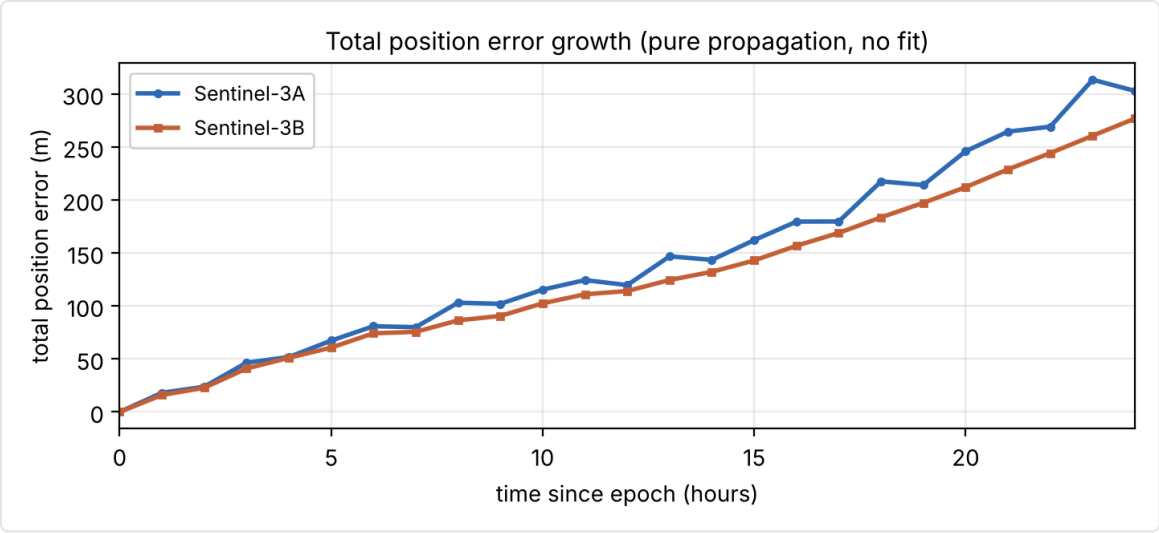


Figure 1. Total position error growth for both satellites (pure propagation, no fit). Near-quadratic growth is the signature of a secular along-track acceleration error (drag).

Table 3. Pure-propagation error (full model). Radial/cross-track stay at the metre level; along-track dominates and grows with time.

Satellite	Span	Total RMS	Total max	Radial (max)	Cross (max)	Along (max)	Wall time
Sentinel-3A	12 h	81.8 m	131 m	~6 m	~28 m	~123 m	0.7 s
Sentinel-3A	24 h	165.6 m	315 m	~10 m	~28 m	~313 m	1.1 s
Sentinel-3B	24 h	146.2 m	277 m	~4 m	~27 m	~277 m	0.6 s

9.2 Orbit determination

Table 4. Batch least-squares over a 6 h arc (120 s observation spacing, gravity 20×20).

Satellite	Estimated	Pre-fit RMS	Post-fit RMS	Improvement	Iters	Est. C_D
Sentinel-3A	state (6)	38.43 m	4.81 m	8.0×	2	—
Sentinel-3A	state + C_D	38.43 m	4.49 m	8.6×	2	5.26
Sentinel-3B	state (6)	41.18 m	5.94 m	6.9×	2	—
Sentinel-3B	state + C_D	41.18 m	5.16 m	8.0×	2	−3.20

> **Key engineering finding.** The state-only fit is robust and well-conditioned (residual drops ~7–8×). The **drag coefficient is not observable over a 6 h arc**: it diverges to 5.26 (S3A) and −3.20 (S3B) — the normal matrix is ill-conditioned in the C_D direction because, over ~4 revolutions, the drag partial is small and nearly collinear with the along-track state partials. The fix is longer arcs and/or an estimated empirical along-track acceleration with proper weighting (§12).

10 Verification: the automated test suite

`mvn test` runs 37 unit tests (all passing), checking physics and numerics against published values, not just internal consistency:

- Loss-free orbit-element conversions and anomaly consistency; two-body energy conservation over 10 orbits.
- Numerical vs analytical Keplerian agreement to sub-metre over an hour.
- J_2 nodal regression matching the secular formula $d\Omega/dt = -1.5\, n\, J_2\, (R_e/p)^2 \cos i$.

- **NRLMSISE-00** reproduces the official reference test case to 7 significant figures.
- **Frames** reproduce SOFA to ~10^{-13} rad; time scales/leap seconds round-trip.
- **DSST** matches two-body exactly, the J_2 secular nodal rate to ~1%, and the Cowell secular SMA decay under drag to ~0.1 m over 3 days (mean-vs-mean).
- **Batch least squares** recovers a perturbed initial state to ~mm and a *known* drag coefficient to within 0.05 from noise-free synthetic observations — confirming the estimator itself is correct (the Sentinel-3 C_D divergence is an observability limit of the short real arc, not an estimator bug).
- **POEORB reader** parses records, state-vector fields and the UT1−UTC offset.

11 Current limitations

Table 5. Known simplifications, each an isolated extension point.

Area	Current state	Impact
Space weather	Constant F10.7/Ap in validation (file-backed provider exists)	Dominant along-track error
Spacecraft geometry	Cannonball (fixed area) for drag & SRP	Ballistic/SRP mismodelling
C_D estimation	Single scalar; unobservable on short arcs	Needs long arc / empirical accel
Sun/Moon ephemeris	Low-precision analytical series (~0.1°)	Small (periodic) radial/cross
EOP	UT1 from file; polar motion zero in validation	Metre-level frame error
Estimation output	No formal covariance; position-only observations	No uncertainty bounds
Jacobian	Finite-difference (re-propagates per parameter)	Slower OD; no analytic STM
TLE seeding	Approximate (no SGP4 recovery)	Coarse TLE-based starts
DSST output	Mean elements (no short-period recovery yet)	Not osculating-accurate

12 Roadmap — what remains and how to improve

Prioritised by accuracy impact per unit effort. P1 highest.

P1 Accuracy of the dominant (drag) channel

- Wire the **file-backed historical space weather** (CssiSpaceWeatherProvider, CelesTrak `SW-All.csv`) into `validate` / `fit` for the exact arc dates — the single biggest expected along-track improvement.
- Estimate a **ballistic scale factor** (or piecewise-constant / empirical along-track acceleration via the existing `EmpiricalAcceleration`) over **multi-day arcs**, resolving the C_D ill-conditioning of §9.2.
- Add a **box-wing** (attitude-dependent area) model for drag and SRP.

P2 Estimation quality

- Replace the finite-difference Jacobian with an **analytical state-transition matrix** (variational equations) — faster, more accurate, and a prerequisite for covariance.
- Emit **formal covariance** and per-parameter sigmas; add observation weighting / data editing.
- Add a **sequential filter** (EKF/UKF) alongside the batch estimator for real-time/streaming use and predictive uncertainty.
- Fit directly to the raw onboard **GNSS observations** (Sentinel-3 GNSSRD product) rather than the reduced POEORB positions.

P3 Conservative-model fidelity & features

- Default geopotential to **70×70**; adopt a **JPL DE** ephemeris for Sun/Moon.
- Add an **IERS EOP file** (finals2000A: polar motion + UT1 time series).
- Solid-tide **Step 2** (frequency-dependent) and pole tide; de Sitter relativity term.
- **DSST short-period recovery** (mean → osculating) for metre-accurate semi-analytical output.
- Proper **SGP4** TLE seeding; additional atmospheres (JB2008, DTM2000).
- Extend validation: full 26 h arcs for both satellites, DSST-vs-Cowell secular comparison, and a density-calibration study once real space weather is wired in.

12b Revised-model update (implemented since first release)

Several roadmap items above are now implemented and validated (see the companion *Model Comparison* report for the full side-by-side numbers):

- DONE **Real space-weather hook** — `validate` / `fit` accept `--sw-file SW-All.csv` (file-backed `CssiSpaceWeatherProvider` with the constant indices as fallback). *Note:* fetching the live CelesTrak file was blocked by the execution environment's network policy, so the real-indices run is pending the host being allow-listed or the file supplied; the plumbing is verified.
- DONE **Earth albedo/IR enabled** in the precise-orbit force config (a sub-metre effect over 24 h at this altitude, as expected).
- DONE **DSST osculating ↔ mean initialization + first-order short-period recovery** (numerical short-period generating function; non-singular vector-based equinoctial conversion). The semi-analytical output is now osculating-accurate.

> **Measured impact (DSST long-term vs a Cowell reference, 30 days, Sentinel-like).** The previous mean-element output drifted to **6,234 km** at day 30 (RMS 3,671 km); the revised engine tracks the osculating truth to **2.76 km** (RMS 1.2 km) — a **3067×** reduction — at ~10× the speed of Cowell. The osculating seed round-trips at epoch to ~0.1 m.

13 Appendix

A Physical constants (as used in code)

Constant	Symbol	Value	Unit
Earth gravitational parameter	μ	3.986004418×10^{14}	$m^3\ s^{-2}$
EGM96 GM	GM	3.986004415×10^{14}	$m^3\ s^{-2}$
Earth equatorial radius	$R_\oplus$	6 378 136.3	m
Earth flattening (WGS84)	f	1/298.257223563	—
Earth rotation rate	$\omega_\oplus$	7.292115×10^{-5}	$rad\ s^{-1}$
Sun GM	$GM_\odot$	$1.32712440018\times10^{20}$	$m^3\ s^{-2}$
Moon GM	$GM_\leftmoon$	4.9028×10^{12}	$m^3\ s^{-2}$
Speed of light	c	299 792 458	$m\ s^{-1}$
Astronomical unit	AU	1.495978707×10^{11}	m
Solar radiation pressure @1 AU	P_0	4.56×10^{-6}	$N\ m^{-2}$
TT − TAI	—	32.184	s

B CLI quick reference

Command	Purpose
`propagate <scenario.properties>`	Propagate a scenario; ASCII plots / CSV
`decay <scenario.properties>`	Orbital-lifetime / re-entry prediction (Monte-Carlo option)
`compare <scenario> --vary k=v1,v2`	Side-by-side model comparison
`validate <poeorb.eof>`	Propagate from a precise state; RTN error vs truth
`fit <poeorb.eof> [--estimate-cd]`	Batch least-squares OD against a precise orbit
`tle <tle-file>`	Parse / propagate a TLE (approximate)